

计算机系统导论实验报告

实验名称: 系统调用分析

班级: 计算 2403

姓名: 钟智勇

日期: 2026 年 1 月 7 日

目录

1 实验报告一：过程调用分析 (IA-32)	3
1.0.1 1. 实验任务与目标	3
1.0.2 2. 实验必备知识	3
1.0.3 3. 实验详细过程记录	3
1.0.3.1 3.1 汇编代码逻辑拆解	3
1.0.3.2 3.2 栈帧状态分析	3
1.0.4 4. 遇到的问题与解决过程	3
1.0.5 5. 实验收获与心得	4

1 实验报告一：过程调用分析 (IA-32)

1.0.1 1. 实验任务与目标

本次实验的核心是“逆向分析”。我需要通过给出的 IA-32 汇编代码，推导出原程序的逻辑结构。具体目标是分析函数 `f` 的参数传递方式、局部变量在栈上的分配、以及它是如何通过递归实现特定算法的，最后用 C 语言还原出源代码。

1.0.2 2. 实验必备知识

- IA-32 栈规范：理解 `%ebp` 始终指向当前栈帧基址，而 `%ebp + 8` 处是第一个入口参数。
 - 寄存器保护策略：区分 Caller-save（调用者保存，如 `%eax`, `%edx`）和 Callee-save（被调用者保存，如 `%ebx`）。在这次实验里，`%ebx` 的入栈备份是理解递归逻辑的关键。
 - 控制流指令：熟悉 `cmpl` 指令如何影响标志位，以及 `jne`（不相等则跳转）如何配合 `jmp` 实现 if-else 分支。
-

1.0.3 3. 实验详细过程记录

我将实验过程分为反汇编逻辑拆解和 C 代码重构两个阶段：

1.0.3.1 3.1 汇编代码逻辑拆解

拿到 08049172 <f> 的反汇编代码后，我重点分析了以下几个关键跳转点：

- 基准情况 A (`n=0`)：在 8049179 处，指令 `cmpl $0x0, 0x8(%ebp)` 将参数与 0 比较。紧接着 `75 07 (jne)` 表示如果就跳走。如果不跳，执行 `mov $0x1, %eax`。这说明当时，返回值是 1。
- 基准情况 B (`n=1`)：跳转后的 8049186 处再次进行比较：`cmpl $0x1, 0x8(%ebp)`。若相等则执行 `mov $0x2, %eax`。由此推断，当时，返回值是 2。
- 递归主体逻辑：如果上述两个条件都不满足，程序进入 8049193。这里两次 `call 8049172 <f>`（递归调用）：

1. 第一次调用前执行了 `sub $0x1, %eax`，即计算。返回值存入 `%ebx`。
2. 第二次调用前执行了 `sub $0x2, %eax`（基于原始的），即计算。
3. 最后在 80491b9 处执行 `add %ebx, %eax`，将两次结果相加。

1.0.3.2 3.2 栈帧状态分析

在分析过程中，我注意到程序在递归前执行了 `sub $0xc, %esp`。通过计算可以发现，这不仅是为了给局部变量留空间，更是为了在 `push %eax` 后，让栈指针满足 IA-32 在调用函数时的 16 字节对齐要求。

1.0.4 4. 遇到的问题与解决过程

- 问题：为什么会有大量的 `sub $0xc, %esp` 和 `add $0x10, %esp`？
- 分析：起初我以为函数内部有很多局部变量，但 C 代码里并没有。
- 解决：后来查阅资料意识到，IA-32 编译器（如 GCC）为了性能，会强制栈顶地址是 16 的倍数。`sub $0xc` 加上后续 `push` 的 4 字节，正好消耗 16 字节，保证了后续 `call` 指令执行时栈是对齐的。
- 问题：`%ebx` 的值去哪了？
- 分析：在代码结尾 80491bb 处有一句 `mov -0x4(%ebp), %ebx`。
- 解决：这是在恢复“被调用者保存”寄存器。因为函数开头 `push %ebx` 把它压到了 `%ebp-4` 的位置，所以退出前必须从这个位置读回来。

1.0.5 5. 实验收获与心得

- 理解了递归的本质：以前觉得递归很玄学，现在从汇编看，递归无非就是不断地压栈、建新栈帧、算结果、弹栈。每一层递归都有自己独立的参数和返回值空间。
- 对寄存器保护有了实感：以前背“调用者保存”和“被调用者保存”总是记混，这次看到程序为了存的结果特意动用了 `%ebx` 并在开头备份，终于明白为什么要分这两类寄存器了。
- 汇编没那么可怕：只要找准 `%ebp+8`（参数）和 `%eax`（返回值）这两个点，再复杂的控制流也能像拆解拼图一样复原出来。